

# Apuntes de IAM

## AWS Certified Solutions Architect – Associate (SAA-C03)

Basado en el curso de Stephane Maarek

### Cómo leer estos apuntes

Los términos clave van en inglés porque así aparecen en el examen. Lo marcado con **[fuera del curso]** es conocimiento estándar de AWS que **no** aparece en las transcripciones del curso.

## Índice

---

|           |                                                        |          |
|-----------|--------------------------------------------------------|----------|
| <b>1</b>  | <b>Qué es IAM</b>                                      | <b>2</b> |
| <b>2</b>  | <b>Root, Users y Groups</b>                            | <b>2</b> |
| <b>3</b>  | <b>Policies</b>                                        | <b>2</b> |
| 3.1       | Estructura (memoriza estas partes) . . . . .           | 3        |
| <b>4</b>  | <b>Password Policy y MFA</b>                           | <b>3</b> |
| 4.1       | Password policy (protege contra brute force) . . . . . | 3        |
| 4.2       | MFA . . . . .                                          | 3        |
| <b>5</b>  | <b>IAM Roles</b>                                       | <b>4</b> |
| <b>6</b>  | <b>Cómo acceder a AWS</b>                              | <b>4</b> |
| 6.1       | Access keys . . . . .                                  | 4        |
| 6.2       | CLI y SDK . . . . .                                    | 5        |
| 6.3       | CloudShell . . . . .                                   | 5        |
| <b>7</b>  | <b>Security Tools (auditoría)</b>                      | <b>5</b> |
| <b>8</b>  | <b>Best Practices</b>                                  | <b>6</b> |
| 8.1       | Hacer . . . . .                                        | 6        |
| 8.2       | Nunca . . . . .                                        | 6        |
| 8.3       | Receta de onboarding (ejercicio 6.1) . . . . .         | 6        |
| <b>9</b>  | <b>Extra: Billing (lección AWS Budget Setup)</b>       | <b>6</b> |
| <b>10</b> | <b>Trampas de examen</b>                               | <b>6</b> |
| <b>11</b> | <b>Tus puntos débiles (repasar en Anki)</b>            | <b>7</b> |
| <b>12</b> | <b>Vocabulario en inglés para tus respuestas</b>       | <b>7</b> |

## 1. Qué es IAM

---

- **IAM = Identity and Access Management.**
- Es un servicio **global**: no tiene región, y lo que creas está disponible en todas.
- Sirve para crear **users**, **groups**, **policies** y **roles**.

## 2. Root, Users y Groups

---

- **Root user**: se crea junto con la cuenta. Úsalo **solo para configurar la cuenta**; después no lo uses ni lo compartas.
- **User**: **1 user = 1 persona física**.
- **Group**: contiene **solo users**, nunca otros groups.
- Un user puede **no** pertenecer a ningún group (posible, pero no es best practice).
- Un user puede pertenecer a **varios groups** (ej.: Charles en *developers* y en *audit*).
- **Least privilege principle**: no des más permisos de los que la persona necesita.

### Del hands-on

- Al crear un user para otra persona: contraseña auto-generada y marcar “*must change at next sign-in*”.
- El curso usa **IAM user**; AWS recomienda Identity Center, pero para el examen, IAM user.
- **Account alias**: acorta la sign-in URL y debe ser único.
- **Tags**: metadata opcional en los recursos.
- Truco: una ventana privada del navegador permite tener root e IAM user abiertos a la vez.
- No pierdas las credenciales de root ni las del admin.

## 3. Policies

---

- Son **documentos JSON** que definen qué se permite hacer.
- Formas de aplicarlas:
  - **Group policy**: aplica a todos los miembros del group.
  - **Inline policy**: solo a un user.
  - **Attach directo** de una managed policy al user (el término “managed” aparece solo en el hands-on).
- Los permisos de un user vienen de **todas las fuentes a la vez**: groups, attach directo e inline.

### 3.1 Estructura (memoriza estas partes)

```
{
  "Version": "2012-10-17", // version del lenguaje de policies
  "Id": "opcional",
  "Statement": [{
    "Sid": "1", // opcional
    "Effect": "Allow", // Allow o Deny
    "Principal": { "AWS": ["arn:aws:iam::123456789012:root"] },
    "Action": ["s3:GetObject"], // llamadas API
    "Resource": ["arn:aws:s3:::example-bucket/*"]
    // "Condition": {...} // opcional
  }]
}
```

- Para el examen, entiende bien **Effect**, **Principal**, **Action**, **Resource**.
- \* significa **cualquier cosa**. Action: \* + Resource: \* es **AdministratorAccess**.
- Wildcards: Get\* o List\* agrupan muchas llamadas (así funciona **IAMReadOnlyAccess**).
- El **policy editor** tiene modo visual y modo JSON.

#### Reglas de evaluación [fuera del curso]

- Todo está denegado por defecto (**implicit deny**).
- Un **Deny explícito siempre gana** sobre cualquier Allow, venga de donde venga.
- Esto **no** depende de si la policy es inline o de group. Error del Bloque 2: razonar “inline > group”.

#### Hands-on que conviene recordar

Al sacar a Stephane del group *admin*, perdió acceso (`iam:ListUsers` denegado). Con **IAMReadOnlyAccess** pudo leer, pero **no** crear un group.

## 4. Password Policy y MFA

### 4.1 Password policy (protege contra brute force)

- Largo mínimo.
- Tipos de caracteres requeridos: mayúscula, minúscula, número, no alfanumérico.
- Permitir o no que el user cambie su propia contraseña.
- **Expiración** (ejemplo del curso: cada **90 días**, no semanal).
- **Prevenir la reutilización** de contraseñas.

### 4.2 MFA

**MFA = algo que sabes (password) + algo que tienes (device)**. Si roban la contraseña, sin el device no pueden entrar. Protege a **root** y a **todos los IAM users**.

| Device                  | Ejemplo                     | Nota                                                  |
|-------------------------|-----------------------------|-------------------------------------------------------|
| <b>Virtual MFA</b>      | Google Authenticator, Authy | Authy soporta varios tokens en un mismo dispositivo   |
| <b>U2F Security Key</b> | YubiKey (Yubico)            | Física; una sola key sirve para varios root/IAM users |
| <b>Hardware key fob</b> | Gemalto                     | Física                                                |
| <b>Key fob GovCloud</b> | SurePassID                  | Solo para AWS GovCloud (US)                           |

Los devices físicos son de **terceros**, no de AWS.

## 5. IAM Roles

- Son como un user, pero **para servicios de AWS**, no para personas.
- Los servicios necesitan hacer acciones *on your behalf* y, para eso, necesitan permisos.
- Ejemplo: una instancia EC2 y su Role forman **una sola entidad**. Cuando la instancia llama a AWS, usa el Role.
- **Roles comunes del curso: EC2 Instance Roles, Lambda Function Roles, CloudFormation Roles.** (S3 y DynamoDB **no** son los ejemplos de la lección.)

### La dirección de los permisos importa

El Role se adjunta **a** la instancia, pero sus permisos son **hacia otros servicios**. Ejemplo: escribir en CloudWatch Logs y leer de S3. No son “permisos para EC2”, son permisos *desde* EC2.

### Por qué usar Role y no access keys [fuera del curso]

- Entrega credenciales **temporales** que AWS rota automáticamente.
- No hay keys estáticas en la instancia que se puedan filtrar.

## 6. Cómo acceder a AWS

| Método                    | Qué es                                              | Qué lo protege              |
|---------------------------|-----------------------------------------------------|-----------------------------|
| <b>Management Console</b> | Interfaz web                                        | Username + password (+ MFA) |
| <b>CLI</b>                | Terminal; comandos <code>aws ...</code>             | <b>Access keys</b>          |
| <b>SDK</b>                | Librerías que <b>embebes en el código</b> de tu app | <b>Access keys</b>          |

### 6.1 Access keys

- **Access Key ID**  $\approx$  username.
- **Secret Access Key**  $\approx$  password.
- Las genera cada user en la consola y se descargan **en ese momento**.
- Son **personales y secretas**: no se comparten nunca.
- Si pierdes el secret, **genera un par nuevo** (no se puede recuperar) y elimina el perdido [fuera del curso].

## 6.2 CLI y SDK

- El CLI es open-source y da acceso a las APIs públicas de AWS. Sirve para scripts y automatización.
- El SDK existe para JavaScript, Python, PHP, .NET, Ruby, Java, Go, Node.js y C++, además de SDKs móviles (Android/iOS) y de IoT.
- El AWS CLI está construido sobre el SDK de Python, **Boto**.

## 6.3 CloudShell

- Terminal gratis dentro de la consola; usa las **credenciales del user logueado**, así que no necesita access keys.
- La región por defecto es donde estás logueado.
- Los archivos **persisten** al reiniciar.
- Permite upload y download de archivos, y varias pestañas y paneles.
- **No está disponible en todas las regiones.**

## 7. Security Tools (auditoría)

| Tool                                         | Nivel         | Qué muestra                                                                                                                                                                      | Para qué sirve                                                             |
|----------------------------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Credentials Report</b>                    | <b>Cuenta</b> | Todos los users y el <b>estado de sus credenciales</b> : password activo, último uso o cambio, MFA activo, access keys (creadas, rotadas, usadas). Se descarga como <b>CSV</b> . | Detectar higiene débil: users sin MFA, keys viejas, passwords sin cambiar. |
| <b>Access Advisor (<i>Last accessed</i>)</b> | <b>User</b>   | Servicios permitidos y <b>cuándo se usaron por última vez</b> , y qué policy dio el acceso.                                                                                      | Quitar permisos sin uso y aplicar least privilege.                         |

### Corrección a tus notas

El Credentials Report **no muestra permisos**: muestra el estado de las **credenciales**. El uso de los permisos lo ve Access Advisor.

### Truco de memoria

**Report** = reporte de **toda** la cuenta (credenciales). **Advisor** = asesor de **un** user (servicios usados).

## 8. Best Practices

---

### 8.1 Hacer

- Crear **un user por persona**.
- Asignar permisos **a groups**, no user por user.
- Crear una **strong password policy**.
- **Exigir MFA**.
- Usar **Roles** para dar permisos a servicios de AWS, incluido EC2.
- Usar **access keys** solo para CLI/SDK.
- Auditar con **Credentials Report** y **Access Advisor**.

### 8.2 Nunca

#### Reglas de “never do this”

1. Usar **root**, salvo para configurar la cuenta.
2. **Compartir IAM users** (si alguien más necesita acceso, créale su propio user).
3. **Compartir access keys**.

### 8.3 Receta de onboarding (ejercicio 6.1)

1. Entra como admin, **no como root**.
2. Crea un user por developer, con password auto-generada y cambio obligatorio.
3. Crea el group *backend-developers* si no existe y agrégalos.
4. Adjunta al group policies de **least privilege**.
5. Verifica que la password policy esté activa.
6. Exige MFA virtual.

## 9. Extra: Billing (lección AWS Budget Setup)

---

- Un IAM user, incluso admin, **no ve la información de billing por defecto**.
- Debe entrar **root**, ir a *Account* y activar “*IAM user and role access to billing information*”.

## 10. Trampas de examen

---

- **Group dentro de group**: no existe; los groups solo contienen users.
- “Necesito dar acceso a una EC2/Lambda” → **Role**, no access keys.
- CLI/SDK → access keys. **Console** → password (+ MFA).
- **Auditar toda la cuenta** → Credentials Report. **Auditar un user** → Access Advisor.
- **Deny explícito gana siempre [fuera del curso]**.
- IAM **no tiene región** (es global).

## 11. Tus puntos débiles (repasar en Anki)

1. Roles: **EC2, Lambda, CloudFormation**.
2. Razón de seguridad de los Roles: credenciales temporales, sin keys estáticas.
3. Access keys: personales, nunca se comparten; si se pierden, se crea un par nuevo.
4. Credentials Report = estado de credenciales, **no** permisos.
5. **SurePassID** = key fob de GovCloud.
6. Password rotation de ejemplo: **90 días**.
7. Da la **razón** de tus respuestas y **nombra ejemplos concretos**.

### Puntaje IAM

**25.5/30 (85%)**, más el refuerzo de Roles y Access Keys (1.3/2).

## 12. Vocabulario en inglés para tus respuestas

| Inglés                       | Español                              |
|------------------------------|--------------------------------------|
| <i>on your behalf</i>        | en tu nombre                         |
| <i>grant / attach a role</i> | otorgar / adjuntar un role           |
| <i>least privilege</i>       | mínimo privilegio                    |
| <i>leaked</i>                | filtrado                             |
| <i>enforce MFA</i>           | exigir MFA                           |
| <i>expire every 90 days</i>  | expira cada 90 días                  |
| <i>an IAM user</i>           | un IAM user (“an” porque suena “ai”) |
| <i>teammate / colleague</i>  | compañero                            |
| <i>“What I mean is...”</i>   | en lugar de “I want to mean”         |

## Ultra-resumen (léelo antes de dormir)

**Users** son personas, **Groups** solo tienen users, **Policies** son JSON (Effect, Principal, Action, Resource), **Roles** son para servicios (EC2, Lambda, CloudFormation), **MFA** más **password policy** protegen el login, **access keys** protegen CLI y SDK, **Credentials Report** audita la cuenta y **Access Advisor** audita un user. Nunca uses root, ni compartas users ni keys.